

LAB: CONCURRENCY CONTROL

Nguyen Ho Ngoc Minh

DS66B

11247201

Lab 1: Experiencing Blocking (Row-Level Lock)

Step 1: Staff 1 acquires an Exclusive Lock (X-Lock)

The image shows two side-by-side PostgreSQL query editors. The left editor, titled 'dvdrental/postgres@PostgreSQL 17', contains a SQL script for Staff 1. The script starts with a comment '-- Staff 1' and '-- Step 1', followed by 'BEGIN;', 'UPDATE film SET rental_rate = 4.99 WHERE film_id = 1;', and a comment '-- Holds X-Lock on film_id = 1'. Below this, there is a comment '-- Step 3 (Run after Step 2 hangs)', 'COMMIT;', and '-- Releases the lock'. The 'Data Output' tab shows 'UPDATE 1' and 'Query returned successfully in 1 secs 313 msec.'. The status bar at the bottom indicates 'Total rows: Query complete 00:00:01.313 CRLF Ln 6, Col 31'.

The right editor, also titled 'dvdrental/postgres@PostgreSQL 17', contains a SQL script for Staff 2. It starts with a comment '--Staff 2' and '-- Step 2', followed by 'BEGIN;', 'UPDATE film SET rental_rate = 5.99 WHERE film_id = 1;', and a comment '--BLOCKS IMMEDIATELY!'. Below this, there is a comment '--Waits for Session A', '-- Step 4 (Observing)', '-- Executes automatically after Acommits.', and 'COMMIT;'. The 'Data Output' tab is empty. The status bar at the bottom indicates 'Total rows: CRLF Ln 11, Col 1'.

Step 2: Staff 2 attempts to update the same row

The query **hangs / freezes**. The loading icon spins continuously, with no output returned.

Query

Query History

1

--Staff 2

2

-- Step 2

3

BEGIN;

4

UPDATE film SET rental_rate = 5.99

5

WHERE film_id = 1;

6

--BLOCKS IMMEDIATELY!

7

--Waits for Session A

8

9

-- Step 4 (Observing)

10

-- Executes automatically after Acommits.

11

COMMIT;

Data Output

Messages

Notifications

Total rows: Waiting for the query to complete... 00:00:08.185 CRLF Ln 5, Col 19

Step 3: Staff 1 releases the lock

Query

Query History

1

-- Staff 1

2

-- Step 1

3

BEGIN;

4

UPDATE film SET rental_rate = 4.99

5

WHERE film_id = 1;

6

-- Holds X-Lock on film_id = 1

7

8

-- Step 3 (Run after Step 2 hangs)

9

COMMIT;

10

-- Releases the lock

Data Output

Messages

Notifications

COMMIT

Query returned successfully in 161 msec.

Total rows: Query complete 00:00:00.161 CRLF Ln 10, Col 21

Query

Query History

1

--Staff 2

2

-- Step 2

3

BEGIN;

4

UPDATE film SET rental_rate = 5.99

5

WHERE film_id = 1;

6

--BLOCKS IMMEDIATELY!

7

--Waits for Session A

8

9

-- Step 4 (Observing)

10

-- Executes automatically after Acommits.

11

COMMIT;

Data Output

Messages

Notifications

UPDATE 1

Query returned successfully in 1 min 28 secs.

Query returned successfully in 1 min 28 secs. X

Q

Total rows: Query complete 00:01:28.978 CRLF Ln 5, Col 19

Step 4: Session B automatically completes

Query	Query History	Query	Query History
<pre> 1 -- Staff 1 2 -- Step 1 3 BEGIN; 4 UPDATE film SET rental_rate = 4.99 5 WHERE film_id = 1; 6 -- Holds X-Lock on film_id = 1 7 8 -- Step 3 (Run after Step 2 hangs) 9 COMMIT; 10 -- Releases the lock </pre>		<pre> 1 --Staff 2 2 -- Step 2 3 BEGIN; 4 UPDATE film SET rental_rate = 5.99 5 WHERE film_id = 1; 6 --BLOCKS IMMEDIATELY! 7 --Waits for Session A 8 9 -- Step 4 (Observing) 10 -- Executes automatically after Acommits. 11 COMMIT; </pre>	
Data Output	Messages	Data Output	Messages
COMMIT		COMMIT	
Query returned successfully in 161 msec.		Query returned successfully in 180 msec.	
Total rows: Query complete 00:00:00.161	CRLF Ln 10, Col 21	Total rows: Query complete 00:00:00.180	CRLF Ln 9, Col 1

Lab 2: Replicating Non-repeatable Read

Step 1:

Query

Query History

1

--Accountant

2

-- Step 1

3

BEGIN;

4

SELECT rental_rate FROM film

5

WHERE film_id = 10;

6

-- Result: 4.99 (S-Lock released)

7

8

-- Step 3

9

SELECT rental_rate FROM film

10

WHERE film_id = 10;

11

--NEWRESULT: 0.99!

12

COMMIT;

Data Output

Messages

Notifications

Showing rows: 1 to 1

Page No: 1 of 1

rental_rate	numeric (4,2)
1	4.99

Total rows: 1

Query complete 00:00:00.150

CRLF

Ln 6, Col 34

Query

Query History

1

--Manager

2

-- Step 2

3

BEGIN;

4

UPDATE film SET rental_rate = 0.99

5

WHERE film_id = 10;

6

7

COMMIT;

8

--Modifies data andcommits

9

--while A is still active.

Data Output

Messages

Notifications

Showing rows: 1 to 1

Page No: 1 of 1

rental_rate	numeric (4,2)
1	0.99

Total rows: 1

Query complete 00:00:00.150

CRLF

Ln 1, Col 10

✓ Successfully run. Total query runtime: 150 msec. 1 rows affected. ✕

Step 2:

Query

Query History

```

1  --Accountant
2  -- Step 1
3  BEGIN;
4  SELECT rental_rate FROM film
5  WHERE film_id = 10;
6  -- Result: 4.99 (S-Lock released)
7
8  -- Step 3
9  SELECT rental_rate FROM film
10 WHERE film_id = 10;
11 --NEWRESULT: 0.99!
12 COMMIT;

```

Data Output

Messages

Notifications

Showing rows: 1 to 1

Page No: 1 of 1

rental_rate	numeric (4,2)
1	4.99

Total rows: 1

Query complete 00:00:00.150

CRLF

Ln 12, Col 8

Query

Query History

```

1  --Manager
2  -- Step 2
3  BEGIN;
4  UPDATE film SET rental_rate = 0.99
5  WHERE film_id = 10;
6
7  COMMIT;
8  --Modifies data andcommits
9  --while A is still active.
10

```

Data Output

Messages

Notifications

COMMIT

Query returned successfully in 121 msec.

Total rows:

Query complete 00:00:00.121

CRLF

Ln 9, Col 27

✓ Query returned successfully in 121 msec. ✕

Step 3:

Query

Query History

```

1  --Accountant
2  -- Step 1
3  BEGIN;
4  SELECT rental_rate FROM film
5  WHERE film_id = 10;
6  -- Result: 4.99 (S-Lock released)
7
8  -- Step 3
9  SELECT rental_rate FROM film
10 WHERE film_id = 10;
11 --NEWRESULT: 0.99!
12 COMMIT;

```

Data Output

Messages

Notifications

COMMIT

Query returned successfully in 99 msec.

Total rows:

Query complete 00:00:00.099

CRLF

Ln 12, Col 8

Query

Query History

```

1  --Manager
2  -- Step 2
3  BEGIN;
4  UPDATE film SET rental_rate = 0.99
5  WHERE film_id = 10;
6
7  COMMIT;
8  --Modifies data andcommits
9  --while A is still active.
10

```

Data Output

Messages

Notifications

COMMIT

Query returned successfully in 121 msec.

Total rows:

Query complete 00:00:00.121

CRLF

Ln 9, Col 27

✓ Query returned successfully in 99 msec. ✕

Lab 03: Creating a Deadlock

Step 1: Session A locks film_id = 100

Step 2: Session B locks film_id = 200

Query	Query History	↗	Query	Query History	↗
1	--Session A		1	--Session B	
2	-- Step 1		2	-- Step 2	
3	BEGIN;		3	BEGIN;	
4	UPDATE film SET rental_rate = 2.99		4	UPDATE film SET rental_rate = 2.99	
5	WHERE film_id = 100;		5	WHERE film_id = 200;	
6			6		
7	-- Step 3		7	-- Step 4	
8	UPDATE film SET rental_rate = 3.99		8	UPDATE film SET rental_rate = 3.99	
9	WHERE film_id = 200;		9	WHERE film_id = 100;	
10	-- Blocks (Waiting for B)		10	--BOOM! ERROR: deadlock detected	

Data Output	Messages	Notifications	↗	Data Output	Messages	Notifications	↗
UPDATE 1				UPDATE 1			
	Query returned successfully in 330 msec.				Query returned successfully in 81 msec.		

Total rows:	Query complete 00:00:00.330	CRLF	Ln 2, Col 1	Total rows:	Query complete 00:00:00.081	CRLF	Ln 5, Col 21
-------------	-----------------------------	------	-------------	-------------	-----------------------------	------	--------------

Step 3: Session A requests the lock on film_id = 200

Query	Query History	↗	Query	Query History	↗
1	--Session A		1	--Session B	
2	-- Step 1		2	-- Step 2	
3	BEGIN;		3	BEGIN;	
4	UPDATE film SET rental_rate = 2.99		4	UPDATE film SET rental_rate = 2.99	
5	WHERE film_id = 100;		5	WHERE film_id = 200;	
6			6		
7	-- Step 3		7	-- Step 4	
8	UPDATE film SET rental_rate = 3.99		8	UPDATE film SET rental_rate = 3.99	
9	WHERE film_id = 200;		9	WHERE film_id = 100;	
10	-- Blocks (Waiting for B)		10	--BOOM! ERROR: deadlock detected	

Data Output	Messages	Notifications	↗	Data Output	Messages	Notifications	↗
				UPDATE 1			
					Query returned successfully in 81 msec.		

Total rows:	Waiting for the query to complete... 00:00:11.069	CRLF	Ln 10, Col 26	Total rows:	Query complete 00:00:00.081	CRLF	Ln 5, Col 21
-------------	---	------	---------------	-------------	-----------------------------	------	--------------

Step 4: Session B requests the lock on film_id = 100 (Deadlock Trigger)

Query	Query History	Query	Query History
<pre> 1 --Session A 2 -- Step 1 3 BEGIN; 4 UPDATE film SET rental_rate = 2.99 5 WHERE film_id = 100; 6 7 -- Step 3 8 UPDATE film SET rental_rate = 3.99 9 WHERE film_id = 200; 10 -- Blocks (Waiting for B) </pre>		<pre> 1 --Session B 2 -- Step 2 3 BEGIN; 4 UPDATE film SET rental_rate = 2.99 5 WHERE film_id = 200; 6 7 -- Step 4 8 UPDATE film SET rental_rate = 3.99 9 WHERE film_id = 100; 10 --BOOM! ERROR: deadlock detected </pre>	
Data Output	Messages	Data Output	Messages
UPDATE 1 Query returned successfully in 48 secs 887 msec.		ERROR: deadlock detected Process 16160 waits for ShareLock on transaction 995; blocked by process 10200. Process 10200 waits for ShareLock on transaction 996; blocked by process 16160. SQL state: 40P01 Detail: Process 16160 waits for ShareLock on transaction 995; blocked by process 10200. Process 10200 waits for ShareLock on transaction 996; blocked by process 16160. Hint: See server log for query details. Context: while locking tuple (73,7) in relation "film"	
Total rows: Query complete 00:00:48.807 CRLF Ln 10, Col 26		Total rows: Query complete 00:00:01.357 CRLF Ln 10, Col 33	

Lab 04: Preventing Race Conditions (FOR UPDATE)

Step 1:

Query

Query History

1

--Customer 1

2

-- Step 1

3

BEGIN;

4

SELECT * FROM inventory

5

WHERE inventory_id = 10 FOR UPDATE;

6

-- Locks the row proactively

7

8

-- Step 3

9

INSERT INTO rental

10

(rental_date, inventory_id, customer_id, staff_id)

11

VALUES (NOW(), 10, 1, 1);

12

COMMIT;

13

Data Output

Messages

Notifications

SQL

Showing rows: 1 to 1

Page No: 1 of 1

inventory_id [PK] integer	film_id smallint	store_id smallint	last_update timestamp without time zone	is_rented boolean
1	10	2	2006-02-15 10:09:17	false

Total rows: 1

Query complete 00:00:00.097

CRLF

Ln 6, Col 29

Query

Query History

1

-- Customer 2

2

-- Step 2

3

BEGIN;

4

SELECT * FROM inventory

5

WHERE inventory_id = 10 FOR UPDATE;

6

-- BLOCKS IMMEDIATELY

7

8

-- Step 4

9

-- Unblocks after A commits.

10

-- App logic detects it 's already rented!

11

ROLLBACK;

Data Output

Messages

Notifications

ROLLBACK

Query returned successfully in 116 msec.

Total rows:

Query complete 00:00:00.116

CRLF

Ln 6, Col 22

Step 2:

Query

Query History

1

--Customer 1

2

-- Step 1

3

BEGIN;

4

SELECT * FROM inventory

5

WHERE inventory_id = 10 FOR UPDATE;

6

-- Locks the row proactively

7

8

-- Step 3

9

INSERT INTO rental

10

(rental_date, inventory_id , customer_id, staff_id)

11

VALUES (NOW() , 10, 1, 1);

12

COMMIT;

13

Data Output

Messages

Notifications

Showing rows: 1 to 1

Page No: 1 of 1

inventory_id [PK] integer	film_id smallint	store_id smallint	last_update timestamp without time zone	is_rented boolean
1	10	2	2006-02-15 10:09:17	false

Total rows: 1

Query complete 00:00:00.097

CRLF

Ln 6, Col 29

Query

Query History

1

-- Customer 2

2

-- Step 2

3

BEGIN;

4

SELECT * FROM inventory

5

WHERE inventory_id = 10 FOR UPDATE;

6

-- BLOCKS IMMEDIATELY

7

8

-- Step 4

9

-- Unblocks after A commits.

10

-- App logic detects it 's already rented!

11

ROLLBACK;

Data Output

Messages

Notifications

Total rows: Waiting for the query to complete... 00:00:02.986

CRLF

Ln 7, Col 1

Step 3:

Query

Query History

1

--Customer 1

2

-- Step 1

3

BEGIN;

4

SELECT * FROM inventory

5

WHERE inventory_id = 10 FOR UPDATE;

6

-- Locks the row proactively

7

8

-- Step 3

9

INSERT INTO rental

10

(rental_date, inventory_id , customer_id, staff_id)

11

VALUES (NOW() , 10, 1, 1);

12

COMMIT;

13

Data Output

Messages

Notifications

COMMIT

Query returned successfully in 430 msec.

Query

Query History

1

-- Customer 2

2

-- Step 2

3

BEGIN;

4

SELECT * FROM inventory

5

WHERE inventory_id = 10 FOR UPDATE;

6

-- BLOCKS IMMEDIATELY

7

8

-- Step 4

9

-- Unblocks after A commits.

10

-- App logic detects it 's already rented!

11

ROLLBACK;

Data Output

Messages

Notifications

Showing rows: 1 to 1

Page No: 1 of 1

inventory_id [PK] integer	film_id smallint	store_id smallint	last_update timestamp without time zone	is_rented boolean
1	10	2	2026-09-25 15:35:49.645298	true

✓ Successfully run. Total query runtime: 15 secs 291 msec. 1 rows affected.

✓ Query returned successfully in 430 msec.

Total rows: 1

Query complete 00:00:00.430

CRLF

Ln 12, Col 8

Total rows: 1

Query complete 00:00:15.291

CRLF

Ln 7, Col 1

Step 4:

Query	Query History		Query	Query History	
1	--Customer 1		1	-- Customer 2	
2	-- Step 1		2	-- Step 2	
3	BEGIN;		3	BEGIN;	
4	SELECT * FROM inventory		4	SELECT * FROM inventory	
5	WHERE inventory_id = 10 FOR UPDATE;		5	WHERE inventory_id = 10 FOR UPDATE;	
6	-- Locks the row proactively		6	-- BLOCKS IMMEDIATELY	
7			7		
8	-- Step 3		8	-- Step 4	
9	INSERT INTO rental		9	-- Unblocks after A commits.	
10	(rental_date, inventory_id , customer_id, staff_id)		10	-- App logic detects it 's already rented!	
11	VALUES (NOW() , 10, 1, 1);		11	ROLLBACK;	
12	COMMIT;				
13					
Data Output	Messages	Notifications	Data Output	Messages	Notifications
COMMIT			ROLLBACK		
Query returned successfully in 430 msec.			Query returned successfully in 128 msec.		
Total rows: Query complete 00:00:00.430	CRLF	Ln 12, Col 8	Total rows: Query complete 00:00:00.128	CRLF	Ln 11, Col 10

✓ Query returned successfully in 128 msec. ✕